

# Lab 6: MPI, and the Deadlock Everyone Writes

Your first MPI programs, then build the classic deadlock on purpose, find the message size at which it appears, and fix it three ways.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

---

**DURATION**

One 2-hour block

**PREREQUISITE**

Unit 2 Part II

**SUBMIT**

lab06-<ROLLNO>.pdf

**MARKS**

12

**OFFERING**

Odd Semester 2026-27

# The point of this lab

---

The symmetric send-then-receive deadlock is written by almost every MPI beginner, and it usually **passes their tests**. This lab has you find the exact message size at which your machine's MPI stops forgiving it.

## Part 0: Ranks and messages (2 marks)

---

**0.1** Write and run the four-line skeleton: `MPI_Init`, `Comm_rank`, `Comm_size`, `MPI_Finalize`, printing rank and size. Run at 1, 4, 7 ranks.

```
mpicc -O2 -std=c11 -Wall -Wextra hello.c -o hello
mpirun --oversubscribe -np 7 ./hello
```

**0.2** Are the lines in rank order? Explain.

**0.3** Send a single `double` from rank 0 to rank 1 with `MPI_Send` / `MPI_Recv` and print it on rank 1. Then deliberately mismatch the tag on the receive. What happens, and why is that worse than an error?

# Part 1: Measure the network (3 marks)

---

Write a **ping-pong** benchmark: rank 0 sends  $n$  bytes to rank 1, rank 1 sends them straight back, repeated  $R$  times. Half the round-trip is the one-way time.

**1.1** Measure for  $n = 8$  bytes up to 8 MB, doubling. Plot time against size on log axes, and bandwidth against size.

**1.2** From the small-message end, read off the **latency**  $\alpha$ . From the large-message end, read off the **bandwidth**  $\beta$ .

**1.3** Fit the model  $T(n) = \alpha + n/\beta$ . At what message size does transfer time equal latency? Compare with your measured crossover.

**1.4** Run it (a) with both ranks on the same node and (b) with the two ranks on different nodes:

```
srun --nodes=1 --ntasks=2 ./pingpong  
srun --nodes=2 --ntasks-per-node=1 ./pingpong
```

Report both  $\alpha$  and  $\beta$ . Explain the difference.

## Part 2: Build the deadlock (4 marks)

```
MPI_Send(sendbuf, count, MPI_DOUBLE, other, 0, MPI_COMM_WORLD);  
MPI_Recv(recvbuf, count, MPI_DOUBLE, other, 0, MPI_COMM_WORLD, &st);
```

Both ranks run this against each other.

2.1 Run with `count = 1`. Does it complete? It probably does.

2.2 Increase `count` by doubling: 1, 10, 100, 1000, ... Find the **exact size range** where it stops completing. Record it.

2.3 Explain what changed. Name the protocol switch and the threshold involved.

2.4 Why is a bug that appears only above a size threshold more dangerous than one that always fails?

### PITFALL · THIS IS THE POINT OF THE WHOLE LAB

Code that deadlocks only above the eager limit passes every small test, gets committed, and hangs in production on a bigger problem. Never rely on buffering: it is an implementation choice, not a guarantee.

## Part 3: Three fixes (3 marks)

---

Implement and verify all three at the size that deadlocked:

**3.1** Order the operations by rank parity. **3.2** `MPI_Sendrecv`. **3.3** `MPI_Irecv` + `MPI_Isend` + `MPI_Waitall`.

**3.4** For 3.3, add real computation between the posts and the wait. Measure with and without the overlap at a large message size. How much of the communication time did you hide?

**3.5** Which fix would you use in production, and why?

# What to submit

lab06-<ROLLN0>.pdf : your two ping-pong plots with  $\alpha$  and  $\beta$  marked, the intra-node versus inter-node comparison, the exact deadlock threshold you found, all three fixes as source, and your overlap measurement.

| MARKS | FOR                                                                           |
|-------|-------------------------------------------------------------------------------|
| 2     | Part 0                                                                        |
| 3     | Part 1: both plots, $\alpha$ and $\beta$ extracted, node comparison explained |
| 4     | Part 2: the threshold found precisely and explained                           |
| 3     | Part 3: three working fixes and a real overlap measurement                    |